Architecture

Autoloads (`src/autoload/`, order matters)

Name	File	Job
`Data`	`data.gd`	Loads every JSON in `data/` once; `Data.balance`, `Data.materials`, `Data.projectiles`, `Data.level(id)`
`Events`	`events.gd`	Global signals only (no logic)
`Save`	`save.gd`	Progress, wallet (`Save.WALLET` = coins, gems; plus `hellfire`) via `amount/add/spend`, settings, purchase ledger `iap`, attempts. `user://save.json`, version 4 (`migrate_v4` converts blueprints)
`Sound`	`sound.gd`	Buses, SFX/UI pools, synced two-stem adaptive music driven by `Events`, stingers, realm ambience + reverb, pause/slow-mo filter. `Sound.play("wood_hit", volume01)`, `set_music`, `set_surroundings` (see `docs/AUDIO.md`)
`Analytics`	`analytics.gd`	Whitelisted events, Crashlytics keys/breadcrumbs, error forwarder. Backend `analytics_firebase.gd` (CastleFirebase plugin) on Android, in-memory log elsewhere
`Ads`	`ads.gd`	Ad rules: `match_played` / `match_break` (one interstitial per 4 matches, shown at a results or pause break), `show_rewarded(id)` for `spin`/`extra_rocky`/`double_coins`, Remove Ads bypass. Backend `ads_admob.gd` (UMP consent first), no-op stub on desktop
`Store`	`store.gd`	BillingManager. Backends `store_play_billing.gd` (GodotGooglePlayBilling 3.3.0) or `store_fake_billing.gd` (tests/debug). Ledger `purchase_processor.gd`, optional server check `purchase_verifier.gd`
`PlayServices`	`play_services.gd`	Google Play In-App Review (after a 3-star win, rules in `balance.play.review`) and In-App Updates (home screen check; release priority ≥ 4 = blocking, else background download installed when the app is backgrounded). CastlePlay plugin in `castle_notify.aar`. No-op off Android
`Notifications`	`notifications.gd`	Local reminders (CastleNotify plugin): plan on pause, cancel on resume, caps, quiet hours. Copy in `data/notifications.json`. Push backend `notifications_push.gd` (GodotX Firebase Messaging): topic subscriptions follow the Settings switch + permission
`Game`	`game.gd`	Scene switching (`Game.goto_level(id)`, `Game.goto_menu()`), command-line flags

Screens

`scenes/main.tscn` is the only scene file. `Game` builds every screen in code: `MainMenu` (src/ui/main_menu.gd) → `WorldMap` (src/ui/world_map.gd) → story scene (a `Level` with `cinematic_mode = true`, played by `Cinematic`) → `Level` (src/game/level.gd). After the fortress flyover the level opens `LoadoutScreen` ("Plan the Assault", a `CanvasLayer` overlay) and stays in INTRO (`Level._planning`) until `confirmed` → `Level.set_heroes(kinds)`; `--no-plan` skips it with `Hud` (src/ui/hud.gd). `UpgradesScreen` (src/ui/upgrades_screen.gd) opens from home, the map and the loadout screen (`Game.goto_workshop(back_to_level)` / `Game.goto_meta("upgrades")`). The other meta screens (spin, daily, shop, skins,

achievements) open with `Game.goto_meta(key, props)`. `SettingsPopup` (src/ui/settings_popup.gd) opens over home and the pause menu. `PromoPopup` (src/ui/promo_popup.gd) is the home feature showcase (an overlay on `UIFxLayer`). `CampaignEnd` (src/ui/campaign_end.gd, `Game.goto_campaign_end()`) follows the last campaign level; `Game.campaign_done()` is true once it is won. `Feats` (src/game/feats.gd) runs the rotating level challenges shown on the Achievements screen.

```
Level
  ├─ GameCamera   frames catapult→fortress, zooms out when a hero flies high
  ├─ Background   realm AI painting softened (downscale + background_haze.gdshader) or SVG fallback,
CanvasLayer -10
  ├─ World        Stage (ledge, fortress bases, gap), catapult, soldiers, blocks, archers, heroes,
arrows, Fx
  ├─ Foreground   realm corners fg_<realm>_left/right (AI) or tinted leaves / fence / bush,
CanvasLayer 1
  └─ Hud          CanvasLayer 10
```

Stage (`src/game/stage.gd`)

Built from the level's `stage` block and the realm's `gap` kind (`balance.json` → `stage`, `gaps`). The catapult ledge (x < `ledge_right`) and each fortress base are static colliders whose top is `y = 0`, so the ground-height rules everywhere else are unchanged. Between them is the gap: water, sea, lava or clouds, drawn as a gradient with scrolling wave strips (one behind, one in front). Bodies crossing `surface_y` splash (`Level.splash`); below `kill_y` blocks break (scored), archers are defeated and heroes finish. Arrows that miss over the gap fall into it (`Stage.ground_at`). Soldiers charge only to the ledge edge. `Stage.top_at(x)` gives the ground height (raised islands) for arrows and the quake shockwave. Moving platforms can carry dressing art (`chassis`, `balloon`) for the boss castles. Terrain art is `stage_ledge_<realm>` and `stage_base_<realm>_wide|small`, placed by its flat top (`Art.stage_top`: SVG 24 px, AI `top_y`).

Cinematics (`src/game/cinematic.gd`)

Story scenes are played inside the real level, not on a text screen. A level's `cinematic` array is a list of beats: `title`, `pan`, `enter` (walk/drop/smoke), `say` (speech bubble over an actor), `emote`, `focus` (camera + ring + arrow + sign), `demo` (ghost hero arc), `hero` (new hero card), `boss` (name plate + roar), `exit`, `wait`, `march`. Targets: `fortress`, `catapult`, `boss`, `top`, `weak`, `gap`, `mech:i`, `captive:i`, `bolt:i` or `"x,y"`. While a scene runs `GameCamera.cine` is true and the HUD is hidden. Tap skips a beat, SKIP ends the scene. Every normal level also opens with a short flyover (fortress pan, up to 3 `callouts`, the boss).

Mechanisms, joints and environment

- `CastleMechanism` (`mechanism.gd`) kinds: gate, drawbridge, moving_platform, swing, spikes, lava, `rope_load` (cut the rope to drop the load, can fire `releases`), `hanging_platform` (1 cut tips, 2 drop), `crusher` (`cycle` or `trigger`), `geyser` (lifts heroes and pieces; lava burns), `frame` (decor). Ropes are cut by hero contact, fire and explosions.
- Phase D kinds: `lever` (sensor at the knob; shots, fast blocks or blasts flip it and fire `triggers`), `windmill` (rotating `AnimatableBody2D`; hub sensor jams it), `shield_ring` (plates on the **shield** physics layer 16, which only projectiles collide with), `rune_switch` (polls `is_covered()` every `rune_switch.check_s`; uncovered it glows and a hit fires `triggers`), `trapdoor` (hinged panel, opened by its `trigger`, wakes what rests on it each step). A trapdoor with `style: rock|timber` becomes a breakable cliff ledge: it tips, then drops away with its load. `flood` owns a `Flood` wave: opening its sluice pushes and floats blocks and drowns covered guards. A `windmill` with `start_on` and `style: wheel` is a jammed water wheel that starts when its brake trigger fires. Blasts, fire, quakes and rune beams call `Level.hit_mechanisms_near()`. Points and float text come from `CastleMechanism.callout_text()` via `Level.on_mechanism_activated(m, pts)`.
- **Rule:** an `AnimatableBody2D` with `sync_to_physics` must get exactly one transform/position assignment per physics frame. Two setters read a stale transform and the body jitters.

- Contraptions (level keys `ramps`, `tethers`):
- `Ramp` (`ramp.gd`): static polygon on the world layer (kinds wood/rock/ice/rail in `balance.ramps`). Hills, fulcrums, rails and slopes for rollers, carts and ice trains.
- `Tether` (`tether.gd`): `pin` (block turns on a hub: seesaws, crane arms), `rope` (rigid pendulum from a hook: bells, cages, wrecking balls; cut by shots, fire and blasts through `Level.cut_ropes_near`, scored by `Level.on_tether_cut`), `spring` (damped spring). A rope with `lock` freezes its block until cut; `kick` then flings it (spring traps). Optional `frame` draws a wooden gantry. Pins can carry `launch` data: when a heavy seesaw turns fast enough, its held payload is thrown with an impact grace period and then becomes a crushing load. Released rope loads and carts damage what they strike; a powder cart detonates its bolted kegs on impact. Built by the `tools/gen_levels.py` helpers (`seesaw`, `dominoes`, `plateau`, `roller`, `bell`, `cage`, `pendulum`, `crane`, `spring_trap`, `rope_bridge`, `launch_seesaw`, `powder_cart`).
- `Bolt` (`bolt.gd`) pins two blocks with two `PinJoint2D`s and snaps on `bolts.break_damage`, an explosion or quake in range, or when either block is destroyed.
- Wind (level `wind`) pushes heroes in flight; the HUD shows the gust. Tide (level `tide`) moves the gap surface, the kill line and floating decks (`Stage.tide_offset`).

Bosses

`boss_unit` in the level JSON makes an `Archer` with `make_boss` (crown, 1.55× scale, more HP, volley fire). Phases fire at HP fractions: `reinforce` (new archers at `reinforce` points), `shield` (bubble), `enrage` (faster fire), `charge` (moving decks speed up through `speed_mult`). The HUD shows a boss bar with phase pips and a callout banner.

Chains and replay

`Chain` (`src/game/chain.gd`, RefCounted) counts links: `Level._chain_link(pos, pts, kind)` is called on block destroyed, archer KO, mechanism trigger, rope cut and bolt snap. `Level._physics_process` calls `chain.tick(now)`; when the window (`chain.window_s`) runs out it returns the bonus, which `_pay_chain` adds to the score. `_finish` pays any open chain before scoring. Milestones drive the HUD chip (`Hud.set_chain`, `chain_callout`), the camera (`GameCamera.set_focus`) and `Level.slow_mo` (skipped in `test_mode`). `Replay` (`src/game/replay.gd`, Node, `PROCESS_MODE_ALWAYS`) samples position, rotation and look of blocks, archers, projectiles and mechanism bodies at `replay.hz`, plus events (`break`, `explode`, `ko`, `splash`). `play(t0, t1, focus)` hides live nodes, draws ghost sprites with letterbox bars and emits `finished`; `stop()` restores the world and camera. The result panel calls `Hud.start_replay`.

Wizard's Workshop (`src/game/workshop.gd`)

Static class. Coin upgrades per hero are stored in `Save`. `Workshop.config(kind)` returns a copy of the hero's projectile data with the upgrades applied (dotted keys such as `explosion.radius`), and every projectile reads its config through it. Costs and steps live in `balance.json` → `workshop`.

Level state machine (`level.gd`)

`INTRO` (soldiers march) → `PLAY` (real time: reload timer, archer timers, end checks) → `WON` / `LOST`. Several heroes can be in the air at once (`Level.projectiles`).

Art pipeline

`Art` (src/game/art.gd) loads `assets/art/ai/cut/<name>.png` when it exists, otherwise the SVG. Characters are positioned from metadata, not from image size: heroes by their body circle (`body_c`, `body_r`), soldiers/archers by their feet point and standing height (`feet_x/y`, `unit_h`). `tools/cutout.py` writes that metadata, so new AI images drop in without code changes.

Damage flow

`Block`/`Archer` are `RigidBody2D`. Each physics step they compute their velocity change minus gravity; `mass × |Δv|` is the impulse they received. Above `damage.min_impulse` it becomes damage (×bonus if a `Projectile` is among the colliding bodies). Scripted pushes (`push()`) are excluded so explosions aren't counted twice. Projectiles report explosions through `Explosion.apply(world, pos, data)` which queries shapes in a radius. Destroyed blocks emit `Events.block_destroyed` and spawn pooled debris. Material extras live in `Block`: `ignite()` routes fire to melting (`melt_dps`), red-hot (`heat`) or burning; `take_damage()` applies the heat multiplier and calls `bend()` (shape swap to `shapes.<id>.bent`, sprite and collider rebuilt, collider "v"). `absorb(kind)` is checked by `Level.explode`, `fire_burst`, `quake` and `rune_beam`; a charged rune calls `Level.rune_beam`, which raycasts along `Block.beam_dir()` and records a `beam` replay event.

Data flow

JSON → `Data` → read by level/blocks/projectiles. Results → `Events.level_finished` → `Save` records stars + coins → `Ads` decides whether an interstitial is due.

Testing hooks

`Game` reads user args after `--`: `--level=N` jumps into a level, `--screenshot=path` saves a frame after `--shot-delay` seconds then quits, `--autoshot=angle,power[,ability_s]` fires the first hero, `--run-tests` runs `tests/run_tests.gd`, `--story=N` plays a story scene, `--workshop` and `--map` open those screens.

Meta UI helpers

- `CurrencyBar` (`src/ui/currency_bar.gd`): wallet pills that roll up and pop on `Events.currency_changed`; `pill_center(kind)` is the target for flying rewards; `deny(kind)` shakes a pill red.
- `UIFx` (`src/ui/ui_fx.gd`, static): `fly_reward`, `count_up`, `press_juice`, `breathe`, `pop_in`, `slide_in`, `pop`, `shake`, `rays`, `confetti`, `stamp`, `celebrate`. Draws on a top `CanvasLayer` ("UIFxLayer", layer 50). Timings in `balance.json` → `ui_fx`.
- `MetaFrame` (`src/ui/meta_frame.gd`): shared meta-screen frame (back, title sign, wallet, rail); `has_news(key)` drives the red dots on home, map and rails; `preview_catapult` shows the real catapult (tinted with `Catapult.apply_tints`) in Upgrades and Skins.
- `MainMenu` is the home screen; `WorldMap` adds realm chest and secret nodes (`_extras`).

Boss rig

`BossRig` (`boss_rig.gd`) is created by `Level` when a level has `power_core` / `cannon_turret` blocks or `engineers` (level JSON `[{x, y}]`). It locks the boss shield (`Archer.lock_shield`) until every core breaks (`Level.on_core_down`), fires cannonballs (`Arrow` with `hits` = 2, art `cannonball`) from upright cannons, and lets engineers (`Archer.make_engineer`, they never shoot) rebuild queued broken blocks at their deck-adjusted home. `status()` feeds `Hud.set_boss_status` (chips for core, armor = `shield_ring` mechanisms, cannon, engineer). Generator helpers: `Fort.core/cannon/engineer`; mechanisms built inside a `deck()` get `follow` = the deck index.

Modes and challenges

- `ModeRules` (`mode_rules.gd`) is made by `Level` from `data.mode` (or `boss_rush` while `Game.rush` is set). Hooks: `setup`, `tick` (in PLAY), `check_end` (can end the level), `blocks_win` (holds the default "all guards down" win), `on_block_destroyed`, `stars`, `chip_text` → `Hud.set_mode`, `result_rows` → results breakdown.

- Level ids above `Data.CHALLENGE_BASE` (100) are challenges: `Data.challenge(id)`, fixed heroes, results in `Save.data.challenges`, and no campaign unlocks. `Data.challenge_list()` contains only ids 101–108 used by `Save.daily_challenge_id()`.
- Master Sieges are special challenge ids 121–130. `Data.special_list()` reads their separate index; `Save.special_pack_unlocked()` requires 25 campaign stars, and `Game.can_enter_level()` enforces the gate. They use normal challenge results/rewards but can never become the daily challenge.
- Reverse siege engines are `moving_platforms` with `approach: {speed, dock_x}`; `sink()` drops them.
- Boss rush: `Game.start_rush()` → `Game.rush_advance()` carries heroes, soldier HP and time between fights; `Save.record_rush()` keeps the best time.
- Screens: `ChallengesScreen` (`challenges_screen.gd`), `--challenges` and `--challenge=N` flags.

Progression

`Progression` (`progression.gd`, static) stores everything in `Save.data`: `stats` (lifetime counters bumped by `Level._stat`), `weapon`, `ups_vanguard` / `ups_weapon`, `owned`, `skin`, `trail`, `realm_chests`, `ach_claimed`, `secrets`. `Level` reads the weapon multipliers (speed, gravity, mass, reload) outside test mode; `Soldier` reads HP and shield chance; `Catapult` and `Fx.trail` read the cosmetic tints. A block with decor `scroll` is the realm's hidden map scroll.

Billing (`store.gd`, `purchase_processor.gd`)

1. `Store.buy(id)` opens the Play sheet (one purchase at a time, `busy_product`).
2. Every purchase update (and `query_purchases` at start/resume) goes to `PurchaseProcessor.normalize` → `process(p, verified)`.
3. `process` grants only `PURCHASED` purchases whose token is not in `Save.data.iap.tokens`, writes the token and the reward to the save **first**, then returns "finish" so `Store` consumes (crates) or acknowledges (`remove_ads`). `mark_finished` records completion.
4. Restart recovery: a token that was granted but not finished is finished again without a second grant. `reconcile_owned` restores Remove Ads from Play's owned list.
5. Pending → message only; cancelled/failed → `purchase_cancelled` / `purchase_failed`.

Second chance and Hellfire (`level.gd`)

Out of heroes with structures standing → `hud.show_second_chance(rocky_available)`. "+1 Rocky" is a rewarded ad once per attempt (`rocky_ad_used`); Hellfire (`use_hellfire`) runs `_hellfire_strike`, which plays `HellfireFX` (`src/game/hellfire_fx.gd`): HUD hidden, letterbox bars, red sky tint, rumble, rising embers and the "HELLFIRE!" title; one giant opener meteor in slow motion, then one meteor per block from front to back (glow, flame tail, ember trail; on impact a flash, shockwave, debris, flames, scorch mark and shake, and the block is destroyed). The finale defeats every archer, clears what is left and emits `done`, and the level wins. Score and chain pop-ups are muted while `hellfire_running`. Timings live in `balance.hellfire`; preview with `--level=16 --hellfire=0.3`. Time attack, one-shot and boss rush fail directly.

Save backup

Every `Save.write()` checks the new file parses, keeps the previous good save as `user://save.json.bak`, then renames the temp file into place. An unreadable `save.json` is kept as `save.json.broken` and the backup is loaded (`save_recovered` counts it). A level opens the pause menu on `NOTIFICATION_APPLICATION_PAUSED` / `FOCUS_OUT`, except while a full-screen ad is open (`Ads.showing()`).

`user://save.json` is the only file in Android Auto Backup (`user_data_backup/allow=true`; rules `cs_backup_rules.xml` / `cs_data_rules.xml` written by `tools/patch_android_template.py`). Ads, Firebase and billing caches are never restored, so IDs are not cloned to a new device.

Intro (`src/ui/intro_screen.gd`)

`Game.boot` shows it when `IntroScreen.due()` (save `intro_version` differs from `application/config/version`): fresh installs and updates. Video `assets/intro/intro.ogv` and sound `intro.wav` are captured unchanged from the studio HTML by `tools/capture_intro.py`; then the title card `assets/intro/title.jpg` (tap or 6 s), then home.